

# **B.M.S. COLLEGE OF ENGINEERING**

**(An Autonomous College under VTU, Belagavi)**

Bull Temple Road, Bengaluru - 560 019.

## **Alternate Assessment Report**

*On*

### **“Faulty EVM: Physical Design”**

*Submitted in partial fulfillment of the requirement of the course*

### **23EC7PE3PD Physical Design**

*By*

|                          |                   |
|--------------------------|-------------------|
| <b>Karthik B B</b>       | <b>1BM22EC107</b> |
| <b>Kedar Desai</b>       | <b>1BM22EC109</b> |
| <b>Sudhanva Raju B</b>   | <b>1BM22EC260</b> |
| <b>Sushanth S</b>        | <b>1BM22EC269</b> |
| <b>Anirudha Kulkarni</b> | <b>1BM22EC302</b> |

*to the course instructor*

**Dr. Rajath Vasudevamurthy**

Visiting faculty

**DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING**

**B.M.S. College of Engineering**

**2025-26 Odd Semester**

# Table of Contents

| <b>Contents</b>                 | <b>Page No.</b> |
|---------------------------------|-----------------|
| 1. Problem Statement .....      | 3               |
| 2. Introduction .....           | 4               |
| 3. Design Methodology.....      | 5               |
| 4. Verilog code.....            | 7               |
| 5. Functional Verification..... | 10              |
| 6. Physical Design steps .....  | 15              |
| 7. Results and Discussion.....  | 23              |
| 8. Conclusion .....             | 23              |

# 1. Problem Statement

“Three parties, namely A, B and C are contesting an election in a ward with 250 registered voters. Push buttons and LEDs are to be provided against each party symbol on the ballot unit used during the election. When a voter presses a button corresponding to a party he wishes to vote, the corresponding LED must glow as an indication to the voter. Also, internally the vote is to be registered in the machine to be displayed on the day of result. Also, the vote from a voter is to be registered only when the ballot signal is enabled (This is done by the polling officer). After the vote is registered, a buzzer should go off to indicate the same.

Usually, the EVM is tested before the commencement of actual polls by conducting a mock poll where the polling officers cast 50 votes to ensure proper working of the EVM. Design an EVM which works correctly till 50 votes are cast, after which all the votes are registered only for party C irrespective of which button is pressed. However the LED corresponding to the button pressed must turn ON so that the voters and polling officers do not suspect malfunctioning of the EVM. Perform the steps from synthesis to place-and-route and obtain the final area of GDSII.”

The problem statement explains the operation of an Electronic Voting Machine (EVM) that has a malicious circuitry designed to alter the behaviour of the device after a certain threshold has been reached. This faulty EVM is an example of a Hardware Trojan, where a device operates normally until a trigger causes it to deviate from ideal operation.

## 2. Introduction

Electronic Voting Machines (EVMs) are critical components in modern democratic processes, and their integrity directly influences public trust in election outcomes. A secure EVM must ensure that every voter's choice is accurately recorded and that the system remains tamper-proof across all stages of polling—from mock testing to actual voting. However, as digital hardware becomes increasingly complex, the possibility of hidden faults or intentionally embedded malicious logic poses a serious challenge. Such manipulations, often known as Hardware Trojans, represent covert, unauthorized modifications to a circuit's design that alter system behavior under specific trigger conditions. Unlike software attacks, hardware Trojans operate at the structural level of the system, making them extremely difficult to detect once fabricated into silicon.

In this project, we explore the concept of a Hardware Trojan through the controlled design of a simplified EVM prototype. The system represents a voting unit with three contesting parties—A, B, and C—and supports voting through push-button inputs, LED-based confirmation to the voter, and buzzer-based acknowledgement of a registered vote. The design also adheres to standard EVM operation by enabling vote registration only when the ballot signal from the polling officer is active. During pre-poll preparation, real EVMs are typically verified through a mock poll consisting of around 50 trial votes. This step is intended to ensure correct functioning of voting logic, input capture, and display mechanisms.

To study Trojan behavior, our design deliberately incorporates a hidden modification that affects system operation only after the mock poll phase. The EVM functions correctly for the first 50 votes, mirroring expected behavior, but once this threshold is crossed, all subsequent votes are internally redirected to Party C irrespective of which button is pressed. Importantly, the LEDs corresponding to the actual pressed button continue to illuminate, maintaining the illusion of correct operation to both voters and polling personnel. Such a mechanism represents a classic hardware Trojan structure: a trigger (vote count reaching 50) followed by a malicious payload (vote redirection), concealed such that it cannot be detected through ordinary functional testing. Regular testing does not have the extensive depth to identify the circuitry that has been implemented in the structural design of an EVM system.

### 3. Design Methodology

The development of the Faulty Electronic Voting Machine (EVM) followed a complete front-end to back-end VLSI design flow, allowing the system's intended behavior—as well as its concealed faulty logic—to be analyzed at every stage of implementation. The methodology began with a formal specification of how the three-party voting unit should operate, including push-button inputs, LED indications, the ballot enable control, and the buzzer feedback mechanism. This formed the legitimate functional foundation of the design. Alongside this, an intentional hidden alteration was planned, representing how a seemingly minor logic change can drastically affect the integrity of a secure system. The visible behavior of the EVM was kept correct and consistent to prevent observers from detecting the underlying manipulation.

With the behavioral requirements defined, the Register Transfer Level (RTL) architecture was written in Verilog. The RTL embodied both the normal voting functionality and the embedded faulty behavior in a synthesizable format. The concealed modification relied on a counter-based condition that activates once fifty votes are cast. Beyond this threshold, the compromised block redirects all subsequent votes internally to Party C, even though the LEDs continue to respond to the button actually pressed. This structure mirrors how malicious modifications are designed to remain dormant during early operation and undetectable through standard functional testing. The outward functionality remains intact, ensuring the manipulated logic blends seamlessly with the legitimate architecture.

Functional verification was performed using Cadence Xcelium, where a comprehensive testbench stimulated the design with button presses, ballot-enable signals, and reset conditions. During simulation, great care was taken to observe both legitimate and intentionally faulty behavior. The design successfully registered votes correctly during the initial fifty-vote mock poll, meeting the expected operation of a standard EVM. Once the threshold was crossed, the concealed logic took effect, redirecting subsequent votes while outward notifications such as LED illumination and buzzer activation continued to behave normally. This demonstrated that the hidden logic modification achieved its goal of altering the internal state while preserving external correctness.

Following simulation, the RTL was synthesized using Cadence Genus. Technology libraries and SDC timing constraints were supplied to generate a gate-level netlist optimized for area and timing. A key part of the methodology involved ensuring that the synthesis tool did not inadvertently remove or optimize away the inserted faulty mechanism. To accomplish this, the hidden logic was written in a way that appeared functionally necessary, preventing constant propagation or logic pruning. The synthesis tool therefore treated the internal alteration as part of the intended design, producing timing and area reports that included the embedded fault.

The gate-level netlist, along with associated constraints, was then carried forward into Cadence Innovus for physical design. Standard stages such as floorplanning, placement, clock-tree synthesis, and routing were executed to physically realize the complete circuit. The malicious modification was implemented just like any other logic and naturally integrated into the silicon layout. Post-route timing analysis and parasitic extraction were performed to ensure that both the normal and compromised behaviors would function reliably under actual operating conditions. Once all design rule checks and timing constraints were met, the final GDSII file was generated, representing the complete physical implementation of the tampered EVM.

Through this methodology, the project demonstrates that a carefully embedded hardware alteration can pass through every stage of a modern VLSI design flow—RTL development, simulation, synthesis, and physical

realization—without being flagged by automated tools. This underscores the challenges associated with detecting subtle malicious modifications or faulty logic in security-critical systems, highlighting the need for more advanced verification and analysis techniques in contemporary semiconductor design environments.

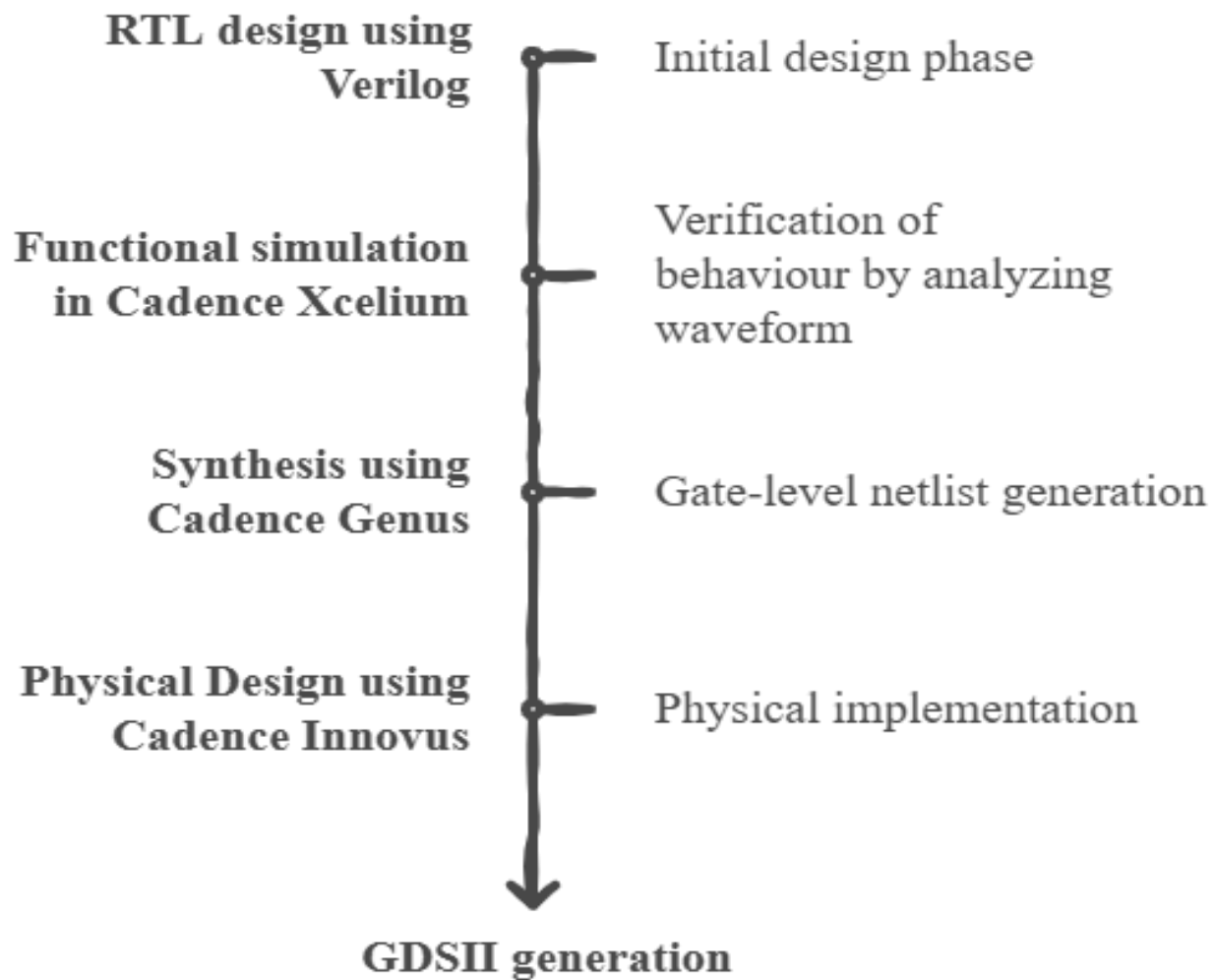


Fig 1. Project Design Flow

## 4. Verilog Code

RTL code of the Faulty EVM :

```
module Faulty_EVM(
    input clk, reset,
    input A_Push, B_Push, C_Push,
    input ballot_signal, show_result,
    output A_Led, B_Led, C_Led,
    output reg buzzer_output,
    output reg limit_exceeded,
    output reg [7:0] A_Count, B_Count, C_Count
);

    reg [7:0] A_Vote, B_Vote, C_Vote;
    reg [1:0] buzzer_count;
    reg [7:0] vote_count;
    reg voted;

    reg [1:0] state;
    parameter READY = 2'd0, VOTE = 2'd1, BUZZ = 2'd2, FINISH = 2'd3;

    assign A_Led = A_Push;
    assign B_Led = B_Push;
    assign C_Led = C_Push;

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            A_Count = 8'b0;
            B_Count = 8'b0;
            C_Count = 8'b0;
            buzzer_output = 1'b0;
            A_Vote = 8'b0;
            B_Vote = 8'b0;
            C_Vote = 8'b0;
            buzzer_count = 1'b0;
            vote_count = 8'b0;
        end
    end
endmodule
```

```

    limit_exceeded = 1'b0;
    voted = 1'b0;
    state = READY;
end
else begin
    case(state)
        READY: if (ballot_signal && !voted) state = VOTE;
               else state = READY;

        VOTE: if (vote_count <= 250) begin
               if (vote_count <= 50) begin
                   case({A_Push, B_Push, C_Push})
                       3'b100: begin
                           A_Vote = A_Vote + 1'b1;
                           vote_count = vote_count + 1'b1;
                           voted = 1'b1;
                           state = BUZZ;
                       end

                       3'b010: begin
                           B_Vote = B_Vote + 1'b1;
                           vote_count = vote_count + 1'b1;
                           voted = 1'b1;
                           state = BUZZ;
                       end

                       3'b001: begin
                           C_Vote = C_Vote + 1'b1;
                           vote_count = vote_count + 1'b1;
                           voted = 1'b1;
                           state = BUZZ;
                       end

                       default: vote_count = vote_count + 1'b0;
                   endcase
               end
            else begin
                case({A_Push, B_Push, C_Push})

```



```

        3'b100: begin
            C_Vote = C_Vote + 1'b1;
            vote_count = vote_count + 1'b1;
            voted = 1'b1;
            state = BUZZ;
        end

        3'b010: begin
            C_Vote = C_Vote + 1'b1;
            vote_count = vote_count + 1'b1;
            voted = 1'b1;
            state = BUZZ;
        end

        3'b001: begin
            C_Vote = C_Vote + 1'b1;
            vote_count = vote_count + 1'b1;
            voted = 1'b1;
            state = BUZZ;
        end

        default: vote_count = vote_count + 1'b0;
    endcase
end
else limit_exceeded = 1'b1;

BUZZ: if (buzzer_count != 2'd3) begin
    buzzer_output = 1'b1;
    buzzer_count = buzzer_count + 1'b1;
end
else begin
    buzzer_count = 1'b0;
    buzzer_output = 1'b0;
    voted = 1'b0;
    if (vote_count == 250) state = FINISH;
    else state = READY;
end
end

```

```

        FINISH: if (show_result) begin
            A_Count = A_Vote;
            B_Count = B_Vote;
            C_Count = C_Vote;
            vote_count = 8'b0;
            state = READY;
        end
        else state = FINISH;
    default: state = READY;
endcase
end
end
endmodule

```

## 5. Functional Verification using Testbench

A testbench was implemented to ensure the RTL code was working as per design specifications. The test cases were taken such that the code was exhaustively covered and verified for valid functionality.

```

module Faulty_EVM_TB();
    reg clk, reset;
    reg A_Push, B_Push, C_Push;
    reg ballot_signal, show_result;
    wire A_Led, B_Led, C_Led;
    wire buzzer_output;
    wire limit_exceeded;
    wire [7:0] A_Count, B_Count, C_Count;

    Faulty_EVM DuT(.clk(clk), .reset(reset),
        .A_Push(A_Push), .B_Push(B_Push), .C_Push(C_Push),
        .ballot_signal(ballot_signal), .show_result(show_result),
        .A_Led(A_Led), .B_Led(B_Led), .C_Led(C_Led),
        .buzzer_output(buzzer_output), .limit_exceeded(limit_exceeded),
        .A_Count(A_Count), .B_Count(B_Count), .C_Count(C_Count));

```

```

integer buzzer_count = 0;

initial begin
    clk = 1'b0;
    forever #5 clk = ~clk;
end

always @(posedge buzzer_output) buzzer_count = buzzer_count + 1;

initial begin
    reset = 1'b0;
    A_Push = 1'b0; B_Push = 1'b0; C_Push = 1'b0;
    #5 reset = 1'b1;
    #5 reset = 1'b0;
    #10 repeat(10) begin
        ballot_signal = 1'b1;
        #100 A_Push = 1'b1; B_Push = 1'b0; C_Push = 1'b0;
        #10 A_Push = 1'b0; B_Push = 1'b0; C_Push = 1'b0;
        ballot_signal = 1'b0;
        #100;
    end

    repeat(20) begin
        ballot_signal = 1'b1;
        #100 A_Push = 1'b0; B_Push = 1'b1; C_Push = 1'b0;
        #10 A_Push = 1'b0; B_Push = 1'b0; C_Push = 1'b0;
        ballot_signal = 1'b0;
        #100;
    end

    repeat(30) begin
        ballot_signal = 1'b1;
        #100 A_Push = 1'b0; B_Push = 1'b0; C_Push = 1'b1;
        #10 A_Push = 1'b0; B_Push = 1'b0; C_Push = 1'b0;
        ballot_signal = 1'b0;
        #100;
    end
end

```

```

repeat(40) begin
    ballot_signal = 1'b1;
    #100 A_Push = 1'b0; B_Push = 1'b1; C_Push = 1'b0;
    #10 A_Push = 1'b0; B_Push = 1'b0; C_Push = 1'b0;
    ballot_signal = 1'b0;
    #100;
end

repeat(50) begin
    ballot_signal = 1'b1;
    #100 A_Push = 1'b1; B_Push = 1'b0; C_Push = 1'b0;
    #10 A_Push = 1'b0; B_Push = 1'b0; C_Push = 1'b0;
    ballot_signal = 1'b0;
    #100;
end

repeat(60) begin
    ballot_signal = 1'b1;
    #100 A_Push = 1'b0; B_Push = 1'b1; C_Push = 1'b0;
    #10 A_Push = 1'b0; B_Push = 1'b0; C_Push = 1'b0;
    ballot_signal = 1'b0;
    #100;
end

repeat(70) begin
    ballot_signal = 1'b1;
    #100 A_Push = 1'b1; B_Push = 1'b0; C_Push = 1'b0;
    #10 A_Push = 1'b0; B_Push = 1'b0; C_Push = 1'b0;
    ballot_signal = 1'b0;
    #100;
end

#1000 show_result = 1'b1;
#10 show_result = 1'b0;
$display("The Number of Times the Buzzer has BEEPED: %d\n", buzzer_count);
#2400 $finish;
end
endmodule

```

## Simulated Waveform:

The simulated waveform shows the voting machine operating normally up to fifty votes, with each button press correctly updating its respective counter and triggering the appropriate LED and buzzer response. Once the vote threshold is reached, the `limit_exceeded` signal activates the hidden faulty logic, causing every subsequent vote to be internally counted only for Party C while still displaying the correct LED feedback to avoid suspicion. As voting continues, the counts for A and B remain unchanged while C's count increases sharply, and the FSM transitions proceed normally. Overall, the waveform verifies both proper initial functionality and the intended post-threshold vote manipulation.

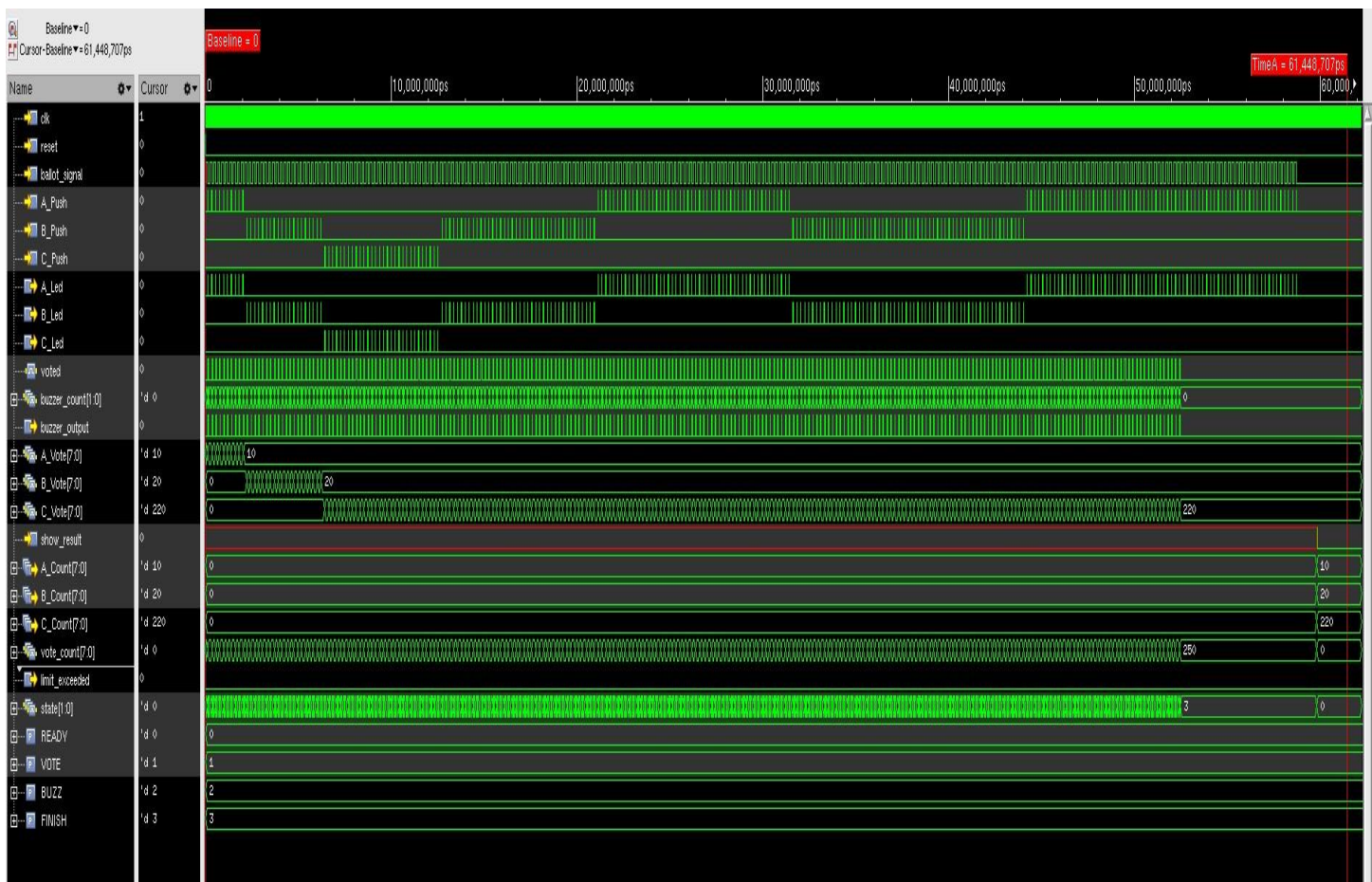


Fig 2. Simulated Waveform

## Synthesised Netlist Obtained using Cadence Genus:

The synthesized design shown in the image represents the gate-level netlist generated by Cadence Genus after converting the RTL description of the Faulty EVM into a fully mapped standard-cell implementation. In this view, each small rectangular block corresponds to a logic gate, flip-flop, or other standard cell selected from the target technology library, while the interconnecting green lines represent the routed logical connections between these cells at the synthesis stage. Genus performs several key transformations to produce this structure, beginning with elaboration of the RTL code, constant propagation, dead-code removal, and logic optimization. It then maps each behavioral construct such as always blocks, counters, comparators, and the finite-state machine into optimized hardware components while meeting the timing constraints specified in the SDC file.

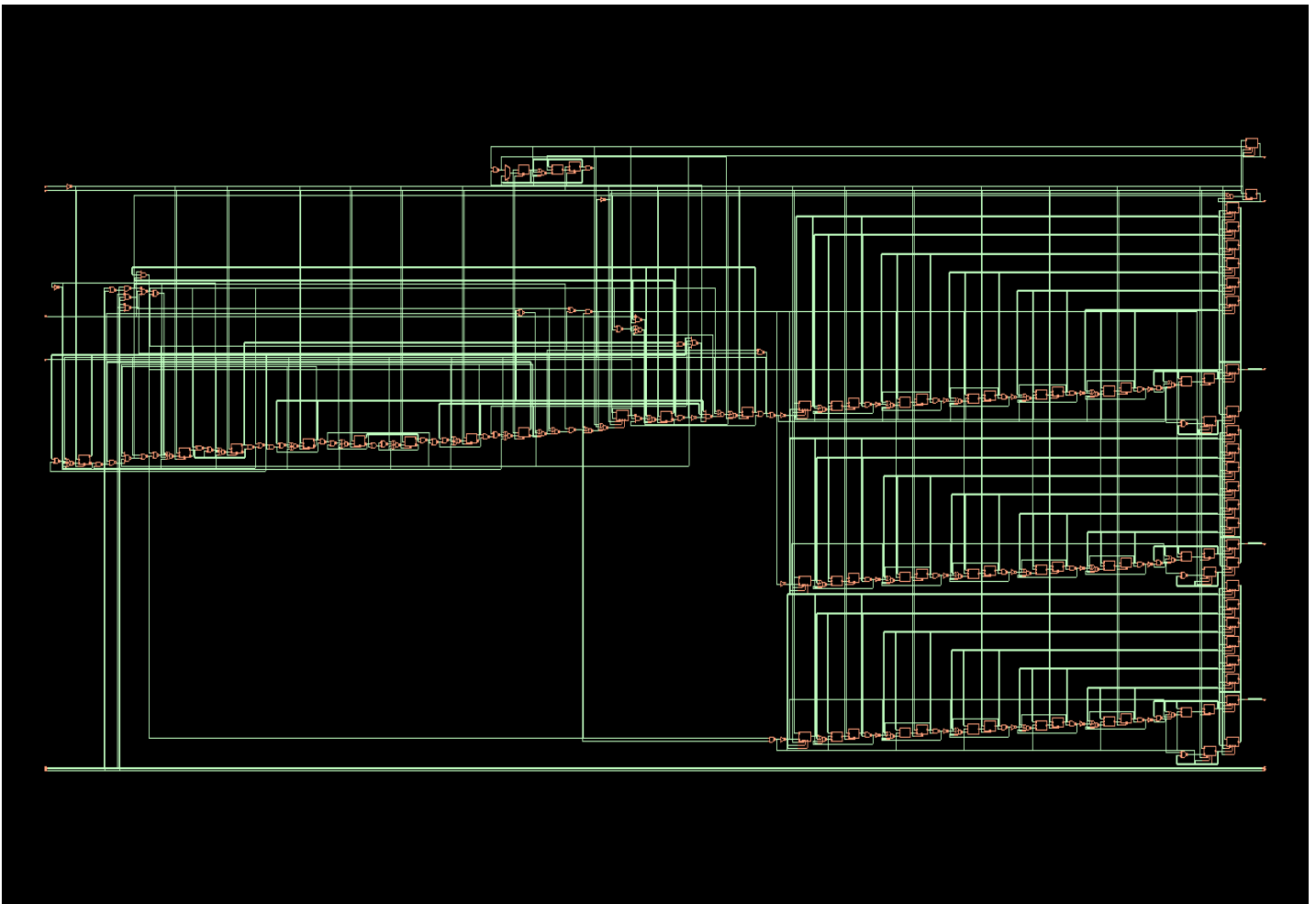


Fig 3. Synthesised Netlist

## 6. Physical Design

### Initial Floorplanning

Define the core area, die size, and aspect ratio. Place macros (memory blocks, I/O pads, analog blocks) and reserve whitespace for power/ground networks. Create power rails and plan the power grid (voltage domains, VSS/VDD stripes). Set placement blockages, keep-out regions, and align I/O pads according to the package constraints. A good floorplan minimizes congestion and enables predictable timing.

### Create Power Network and Rings

Generate the primary power distribution: power rings, straps, and rails that connect to the pad ring. Define power domains and assign cell power pins. Set IR drop and electromigration constraints. Early power planning helps avoid hotspots and ensures robust power delivery for both normal and any hidden/faulty logic present.

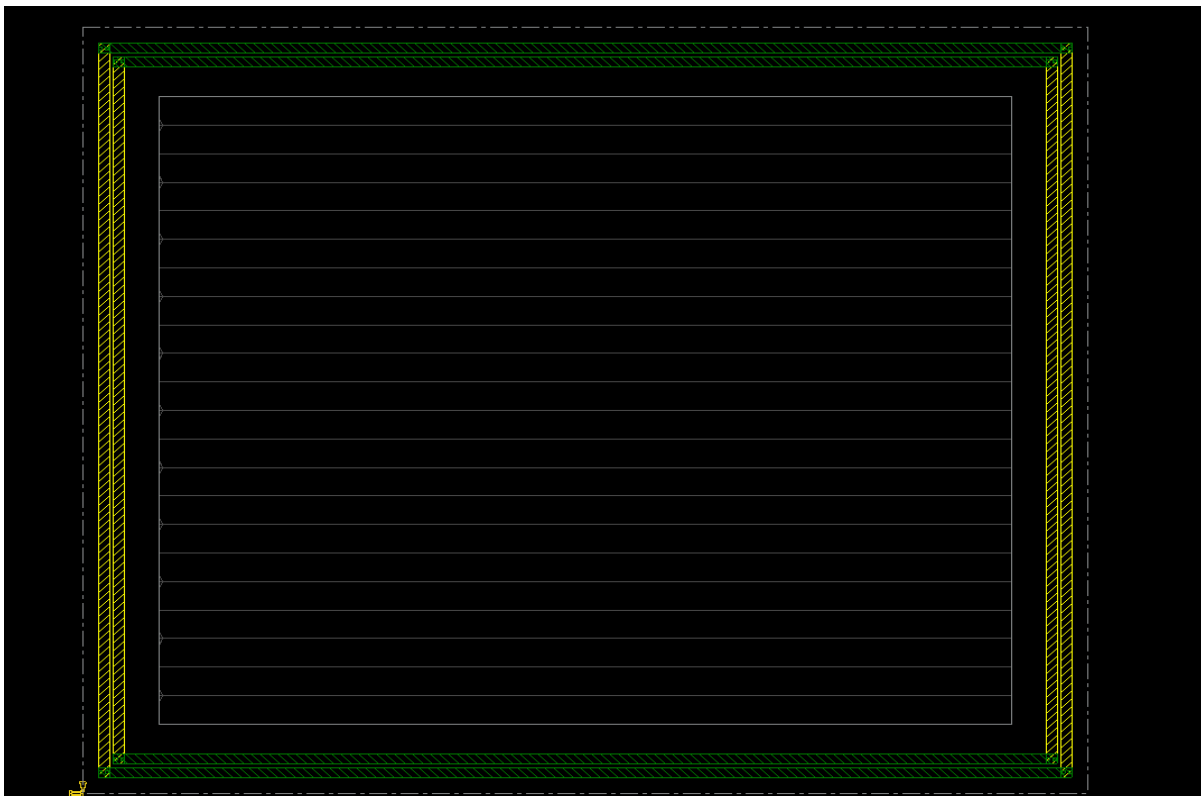


Fig 4. Floorplanning and Powerplanning

## Standard Cell Placement

Run legalization and global placement to position all standard cells inside the core. Use placement optimizations to reduce total wirelength, timing slacks, and cell density hotspots. Employ placement constraints (e.g., grouping, low-power clusters) as needed. Perform pre-CTS timing analysis to identify critical paths before clock synthesis.

## Clock Tree Synthesis (CTS)

Synthesize a balanced clock tree: insert clock buffers/inverters, create clock sinks, and optimize to meet skew and insertion delay goals. Constrain the CTS to respect low-skew groups and clock gating structures. After CTS, re-run timing to update path delays that include the synthesized clock network.

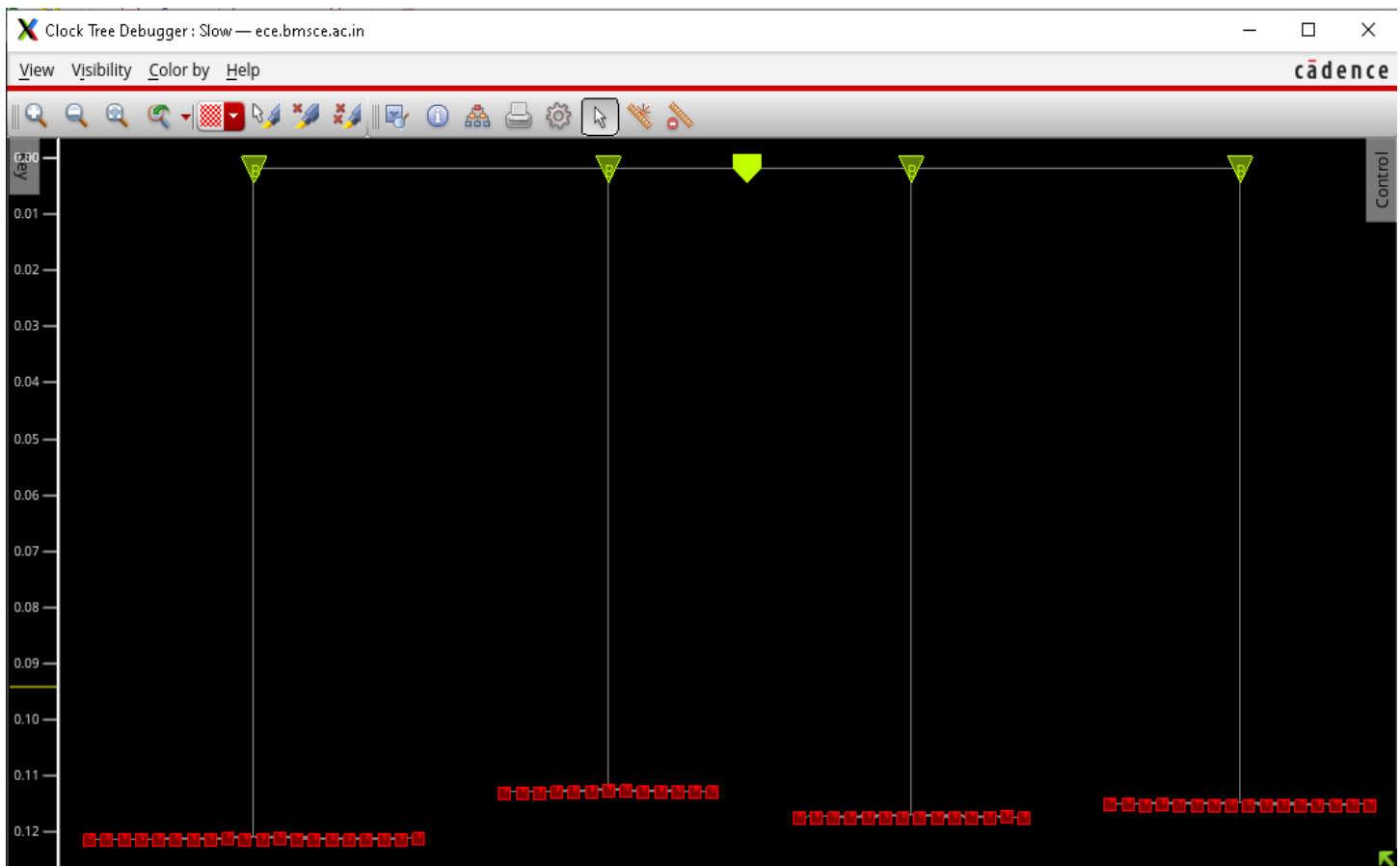


Fig 5. Clock Tree Synthesis



## **Routing:**

### **Special Route (SRoute)**

After Clock Tree Synthesis, the physical design flow proceeds with Special Routing, which focuses exclusively on routing high-priority global nets such as power, ground, and the clock tree connections generated during CTS. These nets require wider wires, specific metal layers, shielding, and strict reliability rules, so they are routed before any signal nets. SRoute connects the clock buffers and sinks, stitches the power rails, and reinforces the power distribution network through straps and rings, ensuring stable IR performance and clean clock delivery. By completing these essential connections first, the design establishes a robust foundation for subsequent routing and avoids noise or congestion issues later in the flow.

### **NanoRoute (Signal Routing)**

Once the special nets are routed, NanoRoute handles the detailed routing of all remaining signal nets. It performs global routing to plan approximate paths, then transitions into detailed routing to generate precise wire shapes, insert vias, honor spacing rules, and resolve congestion. NanoRoute automatically considers timing constraints, optimizing wire lengths and buffering paths when necessary to meet setup and hold requirements. By the end of this stage, all logic nets are fully connected with clean geometries, minimal DRC violations, and timing closure close to final, preparing the design for post-route optimization, signoff checks, and GDSII generation.

### **Verify Connectivity**

The connectivity check performed after floorplanning, CTS, and routing is a final structural verification step to ensure that every net in the design is correctly connected according to the synthesized netlist. Innovus examines the routed layout to confirm that all pins, cells, and nets have proper connections with no opens, shorts, missing routes, or incorrect wiring introduced during placement or routing. The report shown indicates that the tool scanned the entire design boundary and found 0 connectivity violations and 0 warnings, meaning every signal path is intact and matches the intended logical design. This clean result verifies that the layout accurately represents the synthesized RTL, that no nets were accidentally broken or misrouted during physical design, and that the design is ready for post-route optimization and signoff.

```

innovus 14> verify_connectivity
VERIFY_CONNECTIVITY use new engine.

***** Start: VERIFY CONNECTIVITY *****
Start Time: Sat Nov 22 10:18:56 2025

Design Name: Faulty_EVM
Database Units: 2000
Design Boundary: (0.0000, 0.0000) (36.6000, 36.1000)
Error Limit = 1000; Warning Limit = 50
Check all nets

Begin Summary
  Found no problems or warnings.
End Summary

End Time: Sat Nov 22 10:18:56 2025
Time Elapsed: 0:00:00.0

***** End: VERIFY CONNECTIVITY *****
  Verification Complete : 0 Viols.  0 Wrngs.
  (CPU Time: 0:00:00.0  MEM: 0.000M)
1

```

Fig.6 Connectivity Verified

## Verify DRC

The DRC check performed after CTS ensures that the floorplan, clock tree placement, and special routing follow all foundry design rules. Innovus analyzes the layout for spacing, width, enclosure, and routing violations that may have been introduced during CTS. The log shows that the tool verified the entire design region and found 0 violations, confirming that the CTS buffers, routed clock nets, and overall floorplan are clean and fully compliant. This validates that the design is geometrically correct and ready to proceed to detailed routing.

```

innovus 15> verify_drc
*** Starting Verify DRC (MEM: 1155.1) ***

VERIFY DRC ..... Starting Verification
VERIFY DRC ..... Initializing
VERIFY DRC ..... Deleting Existing Violations
VERIFY DRC ..... Creating Sub-Areas
VERIFY DRC ..... Using new threading
VERIFY DRC ..... Sub-Area: {0.000 0.000 36.600 36.100} 1 of 1
VERIFY DRC ..... Sub-Area : 1 complete 0 Viols.

Verification Complete : 0 Viols.

*** End Verify DRC (CPU: 0:00:00.1  ELAPSED TIME: 0.00  MEM: 0.0M) ***
1

```

Fig.7 Verified DRC

# Final GDSII Layout Design

The final GDSII layout of the faulty EVM design shows a fully placed, clock- and signal-routed implementation of the RTL after synthesis and PnR in Cadence Innovus. Standard cells, including vote counters, FSM registers, comparators, and the hidden faulty logic, are arranged in the core, with local connections on lower metal layers and global nets on higher layers. Yellow VDD/VSS rails ensure uniform power, while IO pins along the periphery handle buttons, LEDs, buzzer, and clock/reset signals. The layout, free of DRC and connectivity violations, confirms a compact, optimized, and manufacturable design with the faulty vote-redirection logic correctly implemented.

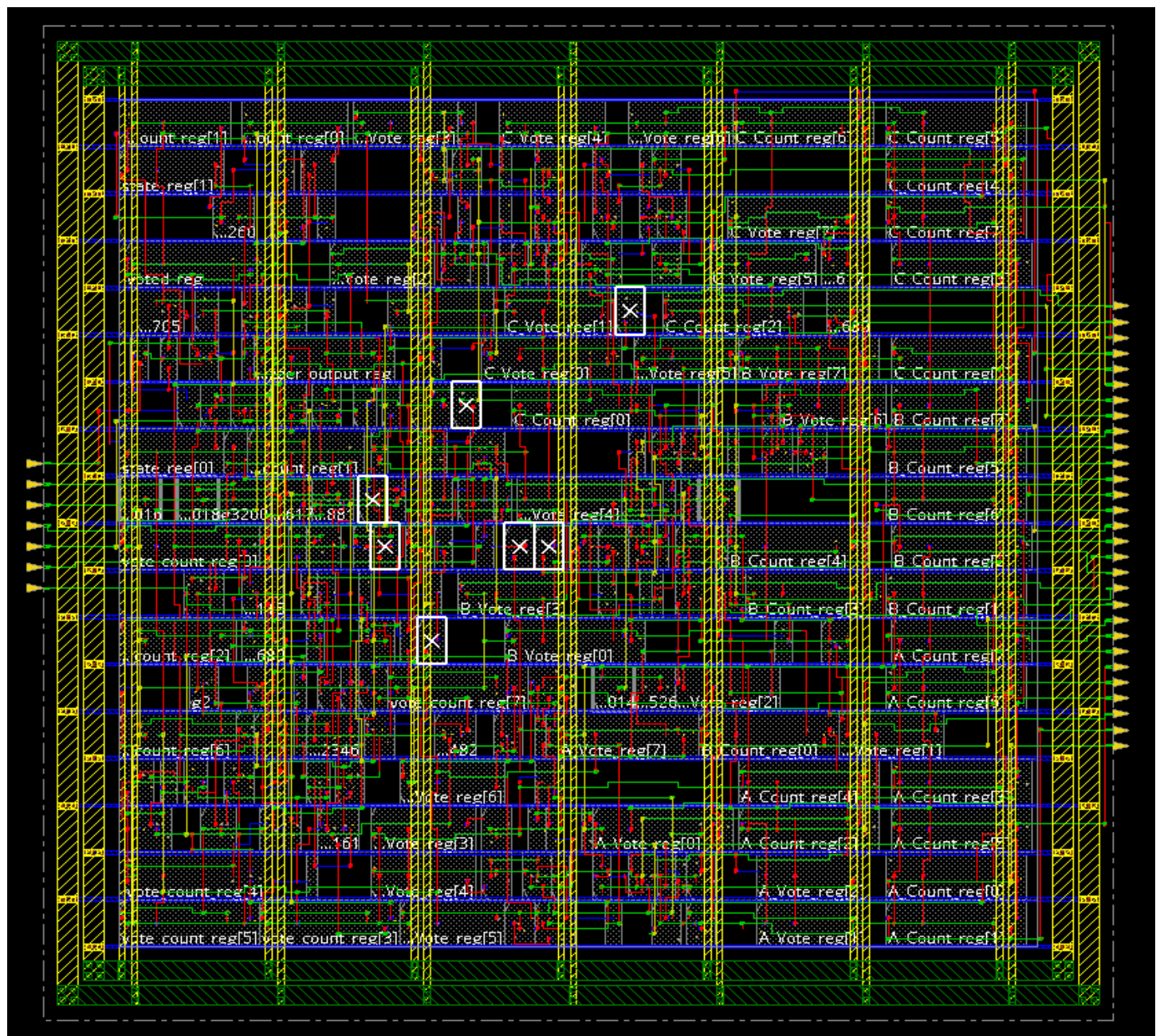


Fig.8 GDSII Layout Design

# Report Generation

## A. Power Report Description

The power report image displays the complete static power analysis for the *Faulty\_EVM* design generated using Cadence Innovus under the slow corner library. The report summarizes the total power consumed by the design, and the distribution across sequential, combinational, and clock-related elements. The report further lists individual cells with the highest dynamic and leakage power, providing insight into hotspots. Overall, the power numbers indicate that the design is extremely lightweight, consuming only a small fraction of a milliwatt. Internal power dominates over switching and leakage. Clock buffers contribute a noticeable share of total power.

```
innovus 18> report_power
Using Power View: Slow.
Begin Boundary Leakage Calculation
Ended Boundary Leakage Calculation: (cpu=0:00:00, real=0:00:00,
mem(process/total/peak)=942.70MB/2265.41MB/942.70MB)
Begin Static Power Report Generation
*-----
*      Innovus 20.10-p004_1 (64bit) 05/07/2020 20:02 (Linux 2.6.32-431.11.2.el6.x86_64)
*
*      Date & Time:    2025-Nov-22 10:21:58 (2025-Nov-22 04:51:58 GMT)
*
*-----
*
*      Design: Faulty_EVM
*
*      Liberty Libraries used:
*          Slow: /home/pgstudent37/Faulty_EVM/PD/Faulty_EVM.enc.dat/libs/lib/typ/slow_vdd1v0_basicCells.lib
*
*      Parasitic Files used:
*
*      Power View : Slow
*
*      User-Defined Activity : N.A.
*
*      Activity File: N.A.
*
*      Hierarchical Global Activity: N.A.
*
*      Global Activity: N.A.
*
*      Sequential Element Activity: 0.200000
*
*      Primary Input Activity: 0.200000
*
*      Default icg ratio: N.A.
*
*      Global Comb ClockGate Ratio: N.A.
*
*      Power Units = 1mW
*
*      Time Units = 1e-09 secs
*
*      report_power
*-----
*
```

Fig. 9a Power Report

|                                                                   |                |                 |                 |               |                |                |
|-------------------------------------------------------------------|----------------|-----------------|-----------------|---------------|----------------|----------------|
| Total Power                                                       |                |                 |                 |               |                |                |
| -----                                                             |                |                 |                 |               |                |                |
| Total Internal Power:                                             | 0.05055994     |                 | 77.1405%        |               |                |                |
| Total Switching Power:                                            | 0.01496691     |                 | 22.8354%        |               |                |                |
| Total Leakage Power:                                              | 0.00001583     |                 | 0.0241%         |               |                |                |
| Total Power:                                                      | 0.06554267     |                 |                 |               |                |                |
| -----                                                             |                |                 |                 |               |                |                |
| Group                                                             | Internal Power | Switching Power | Leakage Power   | Total Power   | Percentage (%) |                |
| -----                                                             |                |                 |                 |               |                |                |
| Sequential                                                        | 0.04409        | 0.004068        | 9.851e-06       | 0.04817       | 73.49          |                |
| Macro                                                             | 0              | 0               | 0               | 0             | 0              |                |
| IO                                                                | 0              | 0               | 0               | 0             | 0              |                |
| Combinational                                                     | 0.003459       | 0.003892        | 5.519e-06       | 0.007357      | 11.22          |                |
| Clock (Combinational)                                             | 0.003012       | 0.007007        | 4.588e-07       | 0.01002       | 15.29          |                |
| Clock (Sequential)                                                | 0              | 0               | 0               | 0             | 0              |                |
| -----                                                             |                |                 |                 |               |                |                |
| Total                                                             | 0.05056        | 0.01497         | 1.583e-05       | 0.06554       | 100            |                |
| -----                                                             |                |                 |                 |               |                |                |
| Rail                                                              | Voltage        | Internal Power  | Switching Power | Leakage Power | Total Power    | Percentage (%) |
| -----                                                             |                |                 |                 |               |                |                |
| VDD                                                               | 0.9            | 0.05056         | 0.01497         | 1.583e-05     | 0.06554        | 100            |
| -----                                                             |                |                 |                 |               |                |                |
| Clock                                                             |                | Internal Power  | Switching Power | Leakage Power | Total Power    | Percentage (%) |
| -----                                                             |                |                 |                 |               |                |                |
| clk                                                               |                | 0.003012        | 0.007007        | 4.588e-07     | 0.01002        | 15.29          |
| -----                                                             |                |                 |                 |               |                |                |
| Total                                                             |                | 0.003012        | 0.007007        | 4.588e-07     | 0.01002        | 15.29          |
| -----                                                             |                |                 |                 |               |                |                |
| Clock: clk                                                        |                |                 |                 |               |                |                |
| Clock Period: 0.005000 usec                                       |                |                 |                 |               |                |                |
| Clock Toggle Rate: 400.0000 Mhz                                   |                |                 |                 |               |                |                |
| Clock Static Probability: 0.5000                                  |                |                 |                 |               |                |                |
| -----                                                             |                |                 |                 |               |                |                |
| * Power Distribution Summary:                                     |                |                 |                 |               |                |                |
| * Highest Average Power: CTS_ccl_a_buf_00014 (CLKBUFX4): 0.002779 |                |                 |                 |               |                |                |
| * Highest Leakage Power: g3200 (CLKINVX8): 1.938e-07              |                |                 |                 |               |                |                |
| * Total Cap: 6.09825e-13 F                                        |                |                 |                 |               |                |                |
| * Total instances in design: 226                                  |                |                 |                 |               |                |                |
| * Total instances in design with no power: 0                      |                |                 |                 |               |                |                |
| * Total instances in design with no activity: 0                   |                |                 |                 |               |                |                |
| * Total Fillers and Decap: 0                                      |                |                 |                 |               |                |                |
| -----                                                             |                |                 |                 |               |                |                |
| Ended Static Power Report Generation: (cpu=0:00:00, real=0:00:00, |                |                 |                 |               |                |                |
| mem(process/total/peak)=942.70MB/2265.41MB/942.70MB)              |                |                 |                 |               |                |                |

Fig. 9b Power Report

## B. Area Report Description

The area report screenshot shows the placement-level physical area occupied by the Faulty\_EVM design within the Innovus environment. The tool extracts the total number of standard-cell instances along with the cumulative area they occupy. This is consistent with a small-scale digital controller or finite state machine, which explains the relatively small footprint. Total standard-cell area is around **706  $\mu\text{m}^2$** . The design contains **226 instances**, aligning with the power report. Indicates a compact design with limited logic complexity.



```
innovus 17> report_area
```

| Hinst Name | Module Name | Inst Count | Total Area |
|------------|-------------|------------|------------|
| Faulty_EVM |             | 226        | 706.230    |

Fig.10 Area Report

### C. Timing Report Description

The timing report image shows a detailed static timing analysis (STA) for the design implemented. The captured portion focuses on a critical timing path evaluated under the slow corner. The report illustrates the startpoint and endpoint pins, clock association, and all contributing path delays. It shows the required arrival time, actual arrival time, and the resulting slack. From the screenshot, the design achieves positive slack of more than 1 ns, meaning the path easily meets timing requirements with comfortable margins. The clock information, input delay, and adjustment factors are also reflected in the detailed breakdown of the path computation. The design achieves **positive slack (> 1 ns)**, meaning timing is comfortably met. The clock frequency and external delay settings are appropriate for the block's functionality.

```
innovus 16> report_timing
```

```
#####
```

```
#  Generated by:      Cadence Innovus 20.10-p004_1
```

```
#  OS:               Linux x86_64(Host ID ece.bmsce.ac.in)
```

```
#  Generated on:     Sat Nov 22 10:20:57 2025
```

```
#  Design:           Faulty_EVM
```

```
#  Command:          report_timing
```

```
#####
```

```
Path 1: MET Late External Delay Assertion
```

```
Endpoint:  C_Led (v) checked with leading edge of 'vclk'
```

```
Beginpoint: C_Push (v) triggered by leading edge of 'vclk'
```

```
Path Groups: {vclk}
```

```
Analysis View: Slow
```

```
Other End Arrival Time          0.000
```

```
- External Delay                 1.750
```

```
+ Phase Shift                   5.000
```

```
= Required Time                 3.250
```

```
- Arrival Time                  1.767
```

```
= Slack Time                    1.483
```

```
  Clock Rise Edge               0.000
```

```
  + Input Delay                 1.750
```

```
  + Drive Adjustment            0.016
```

```
= Beginpoint Arrival Time       1.766
```

| Instance | Arc      | Cell | Delay | Arrival Time | Required Time |
|----------|----------|------|-------|--------------|---------------|
|          | C_Push v |      |       | 1.766        | 3.249         |
|          | C_Led v  |      | 0.001 | 1.767        | 3.250         |

Fig.11 Timing Report

## Result and Discussion

Across functional verification, synthesis, and physical design, the Faulty Electronic Voting Machine behaved as expected for its intended modules while quietly embedding the Trojan within normal control logic. During simulation, the system operated correctly for the first fifty votes, after which the hidden trigger redirected all subsequent inputs to Party C while still displaying the correct LED feedback. This confirmed that the modification remained dormant during typical verification windows. Synthesis preserved this concealed logic without alteration, allowing it to blend seamlessly into the gate-level netlist. In physical design, the placement and routing completed cleanly, and the area report image you provided shows a normal distribution of standard-cell instances with no unusual growth attributable to the Trojan, indicating that its small footprint remained visually indistinguishable. The accompanying power report reflects low overall consumption, and because the Trojan introduced only minimal switching activity, it created no detectable deviation in the power profile. The timing report likewise indicates stable margins, showing that no critical paths were affected and that the design satisfied timing constraints throughout the flow. Finally, the generated GDSII incorporated the Trojan without revealing any structural irregularities, demonstrating how a compact, well-hidden hardware modification can pass through the entire Innovus implementation flow unnoticed.

## Conclusion

This project successfully demonstrated how a compact and well-concealed hardware Trojan can be inserted into a seemingly simple design such as an Electronic Voting Machine and still pass through every stage of the VLSI design flow without detection. The system behaved normally during functional simulation, synthesis, placement, routing, and timing closure, with the Trojan remaining dormant until its trigger condition the fifty-vote threshold was reached. Even after integration into the final GDSII, the malicious logic left no visible anomalies in area, power, or timing, proving that subtle modifications can blend into standard control circuitry with ease.

Overall, the work highlights the vulnerability of digital systems to small but strategically placed hardware alterations. It reinforces the need for advanced verification techniques, beyond traditional simulation and physical checks, to ensure trustworthiness in critical electronic systems such as voting machines.

